

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

30. April 2026

Übung Projekt zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 28 Mai, 2026 10:15** via EPICS: <https://ep.ics.jku.at>

Im Rahmen des Projekts soll der in der Vorlesung vorgestellte RISC-V Single-Cycle Prozessor erweitert und in VHDL realisiert werden.

Für das Projekt steht in Moodle ein Template des Prozessors zur Verfügung, d.h. der Prozessor setzt bereits die in der Vorlesung vorgestellten Instruktionen um. Die erweiterten VHDL Dateien müssen vor der Abgabe in EP.ICS mit *ghdl* ausgeführt werden. Dateien, die nicht kompilieren, können nicht abgegeben werden.

Aufgabe 1 – Konzeptionelle Erweiterung des Prozessors (14 Punkte)

Der in der Vorlesung vorgestellte Single-Cycle RISC-V Prozessor soll erweitert werden, damit er einen Algorithmus zum Summieren eines Byte Arrays unterstützt. Hierzu müssen die Instruktionen **jalu**, **auipc** und **lbu** implementiert werden. **Erweitere** dazu in **Abbildung 2** für jeden dieser Befehle (falls nötig) den Datenpfad und füge weitere Kontrollsignale und Register hinzu¹. Gib für jeden dieser Befehle auch den Wert aller Kontrollsignale an. Nimm dabei an, dass die ALU aus **Abbildung 1** verwendet wird und gib **don't cares** explizit mit **X** an.

Die Abgabe soll ein Bild mit dem erweitertem Daten- und Kontrollpfad des Prozessors pro zusätzlichem Befehl (es können auch mehrere Erweiterungen in ein Bild gezeichnet werden) sowie eine Tabelle mit den Werten aller Kontrollsignale für die neuen Befehle beinhalten. Hinweis: Entnimm die Semantik der einzelnen RISC-V Befehle der in Moodle zur Verfügung gestellten RISC-V Reference Card².

Aufgabe 2 – Implementierung in VHDL (24 Punkte)

Implementiere den erweiterten Single-Cycle RISC-V Prozessor in VHDL. Verwende dazu die im Template vorgegebenen Strukturen und füge (wenn nötig) weitere Komponenten hinzu. Durch das Erweitern des Prozessors kann es sein, dass sich die Schnittstelle einiger Komponenten ändert.

¹Der in **schwarz** gezeichnete Teil in **Abbildung 1** und **Abbildung 2** visualisiert den **Datenpfad**, während der **Kontrollpfad** in **Blau** gezeichnet ist.

²<https://moodle.jku.at/mod/resource/view.php?id=12898432>

Um den erweiterten Prozessor zu testen, gibt es ein Beispielpogramm in `code/sum_bytes.s`. Das Programm ladet das Array `[10,240,120,200,180,50,170,210]` und summiert es Byte für Byte.

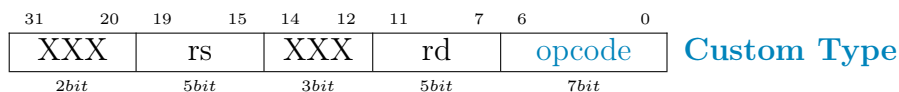
Um das Programm mit deinem Prozessor auszuführen, verwende den schon kompilierten Maschinencode in `files/text.txt` mit den Daten in `files/data.txt`. Nutze dafür die *Makefile*, welche das Programm und den Datenspeicher mit den jeweiligen Dateien befüllt und so den Prozessor auf die Ausführung vorbereitet. Eine Testbench zum Simulieren ist auch im Template enthalten, welche den Prozessor treibt und Print/Exit ECalls behandelt. Verwende `make`, um den Prozessor zu starten!

Am Schluss wird der Wert ausgegeben. Kontrolliere, ob die richtige Zahl 1180 ausgegeben wird.

Hinweis: Bei der Simulation in EP.ICS müssen auch die `.txt` Programmdateien ausgewählt werden. Zusätzlich kann es Sinn machen, eigene kleine Testprogramme zu schreiben. Diese können mit der "Dump" Funktion im Venus Simulator in eine hexadezimale Darstellung übersetzt werden, die durch den Single-Cycle Prozessor ausgeführt werden können.

Aufgabe 3 – Integration einer Custom Instruction (10 Punkte)

Die Implementation in der vorherigen Aufgabe verwendet eine Addition pro Byte im Array. Nun soll dieses Addieren der Bytes durch das Hinzufügen der *Custom Instruction* `bytesum` beschleunigt werden. Die Instruktion hat folgendes Format:



`bytesum` lest ein 32 Bit Registerwort, teilt es in 4 Bytes (unsigned) und speichert deren Summe in einem Register ab:

$$rd = \text{ZeroExt}(rs_{[7:0]}) + \text{ZeroExt}(rs_{[15:8]}) + \text{ZeroExt}(rs_{[23:16]}) + \text{ZeroExt}(rs_{[31:24]})$$

Der *opcode* der Instruktion ist 0001011. "XXX" beschreibt die *don't care* Teile der Instruktion und sollen beim Dekodieren ignoriert werden.

Zum Testen der Instruktion kann der Programmcode in `text_custom_instruction.txt` verwendet werden. Dazu direkt in der Makefile den Pfad ändern oder `make TEXT_FILE=files/text_custom_instruction.txt` ausführen. Das Programm verwendet die neue Instruktion und sollte dasselbe Ergebnis (1180) wie in Aufgabe 2 liefern. Hinweis: Für die Simulation in EP.ICS muss `files/text_custom_instruction.txt` als `text.txt` abgegeben werden.

Aufgabe 4 – (Bonus) BLT Instruktion

(2 Punkte)

Finde heraus, wie die `blt` Instruktion für den Prozessor ICS-NEX-RV32I-SC funktioniert. Verwende dafür SonicRV (<https://sonic-rv.ics.jku.at/simulate>), wähle den Prozessor aus und schreibe ein Assembly Programm zum Ausprobieren der `blt` Instruktion. Notiere dann den Ablauf der Instruktion, i.e. wie die Kontrollsignale gesetzt werden, welche Werte berechnet werden und welche Komponenten dafür notwendig sind.

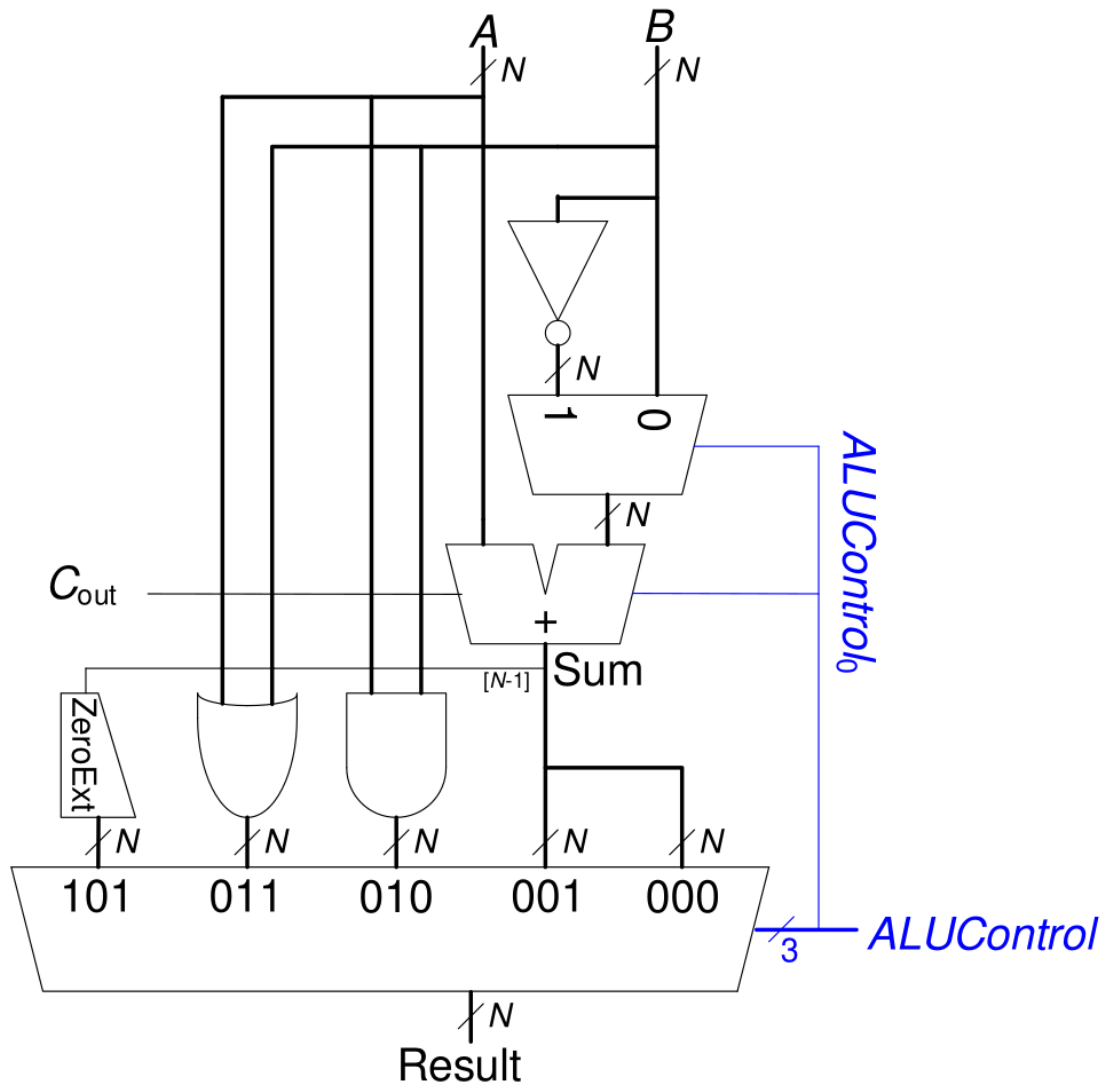


Abbildung 1: ALU des Single-Cycle RISC-V Prozessors

GESAMT ERWEITERUNG

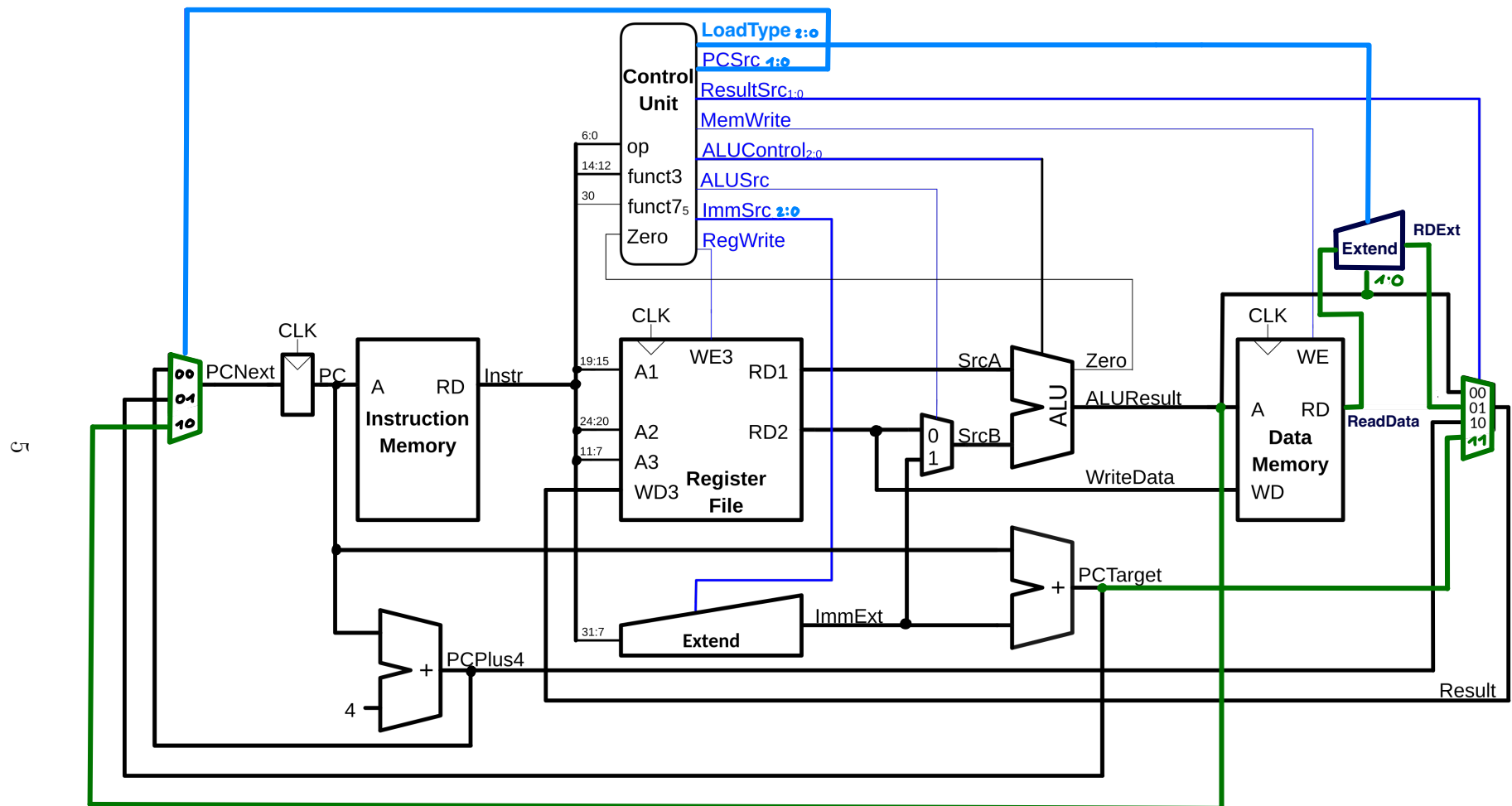


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

INSTRUCTION	Typ	opcode	func3	func7	OPERATION
jalu : rd, rs1, imm	I	1100111	000	—	rd=PC+4, PC=rs1+imm

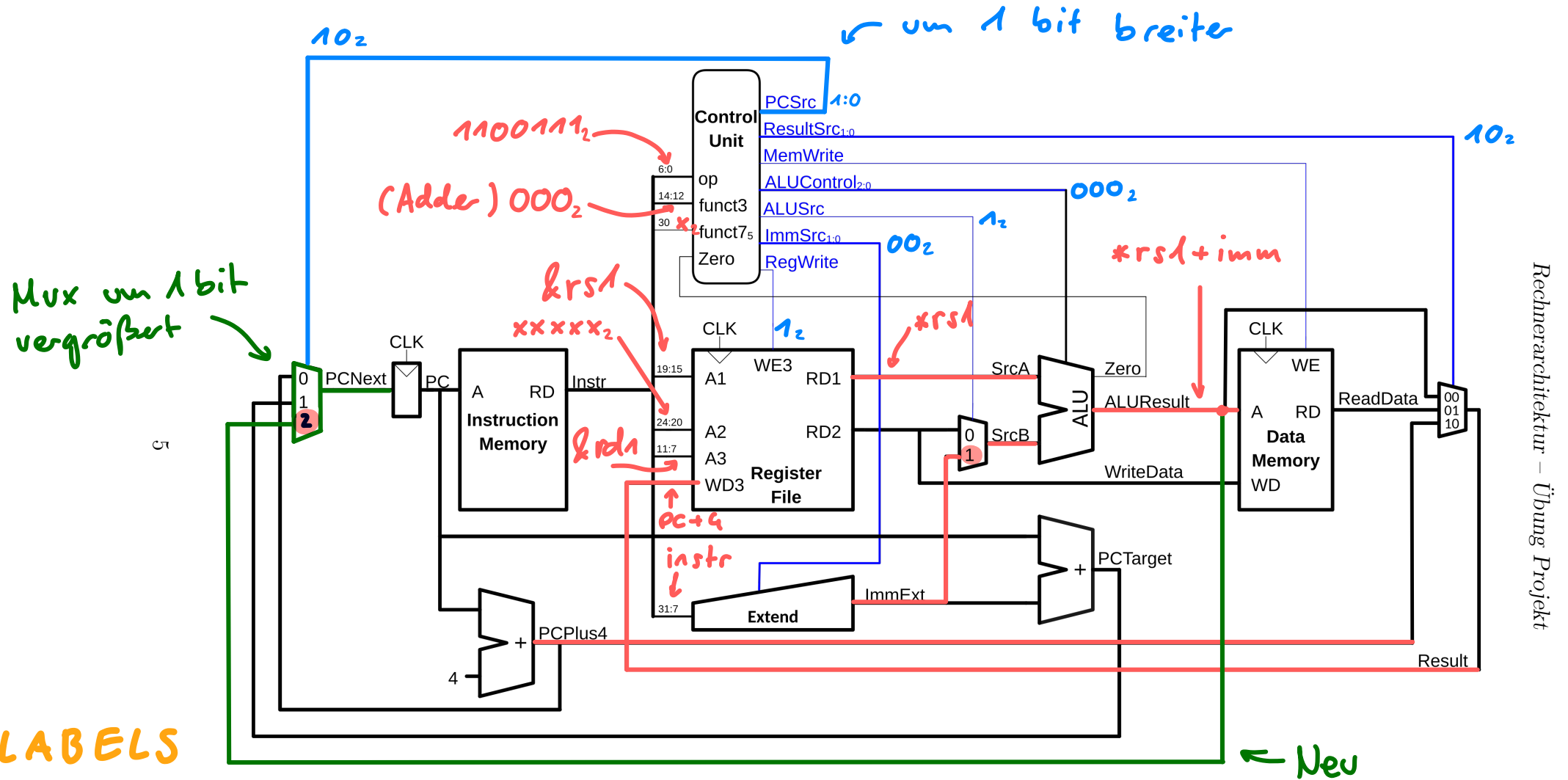
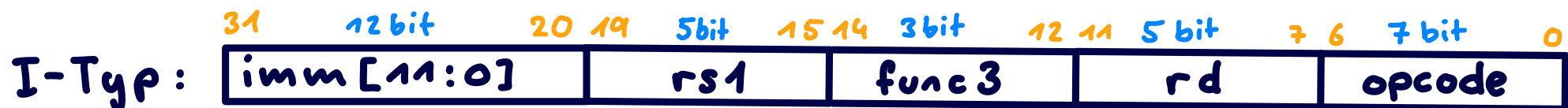


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

- X ... Don't Care
- ® ... Register Adresse
- *reg ... Register Wert

instr = imm[11:0], 20 × X



INSTRUCTION	Typ	opcode	func3	func7	OPERATION
lbu : rd, imm(rs1)	I	0000011	100	—	$rd = \text{ZeroExt}(M[rs1 + \text{imm}]_{7:0})$

Block zur bit Markierung eines Bytes aus 32-Bit Register

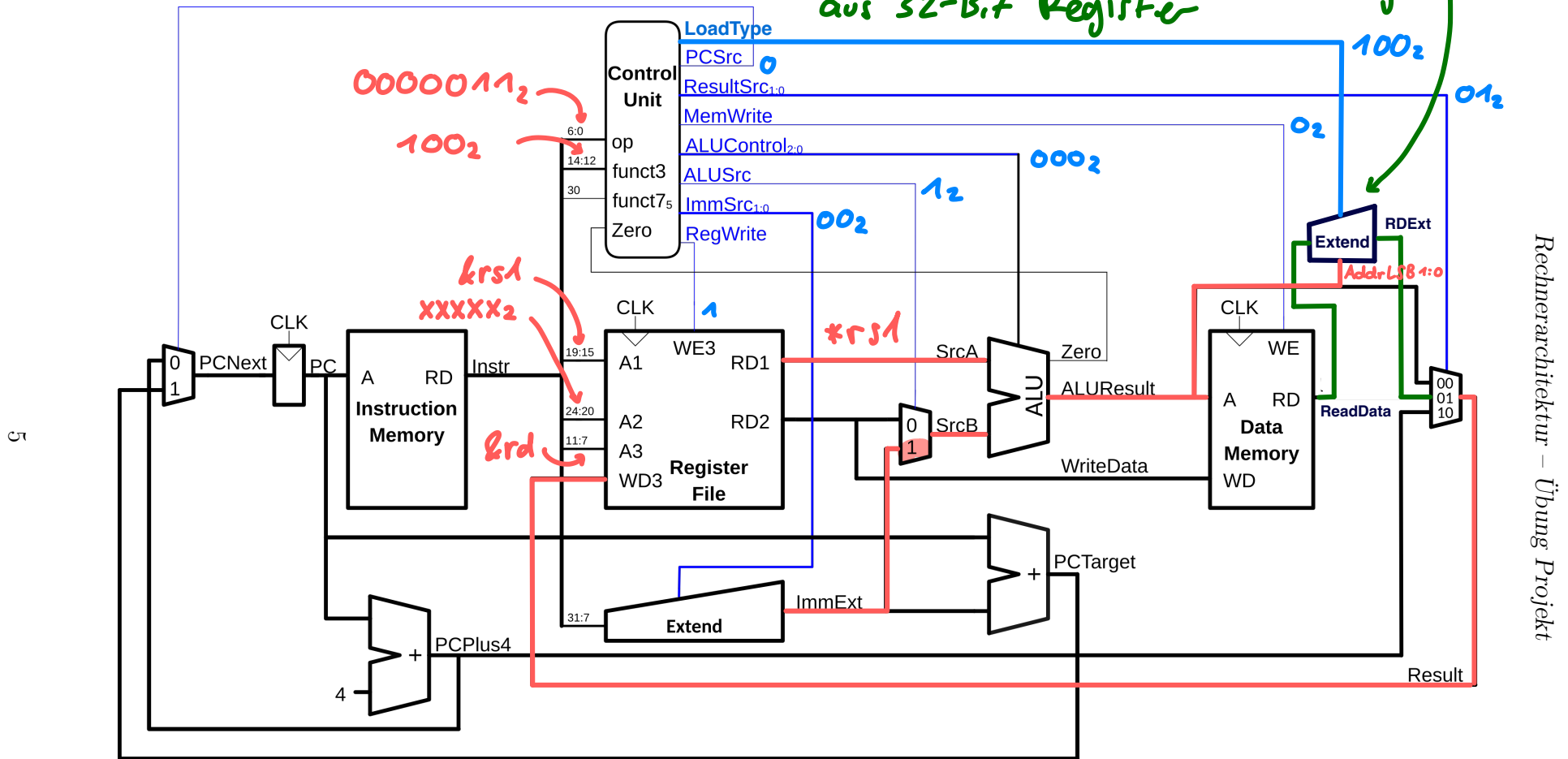
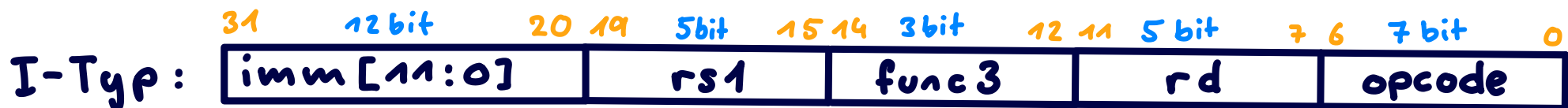
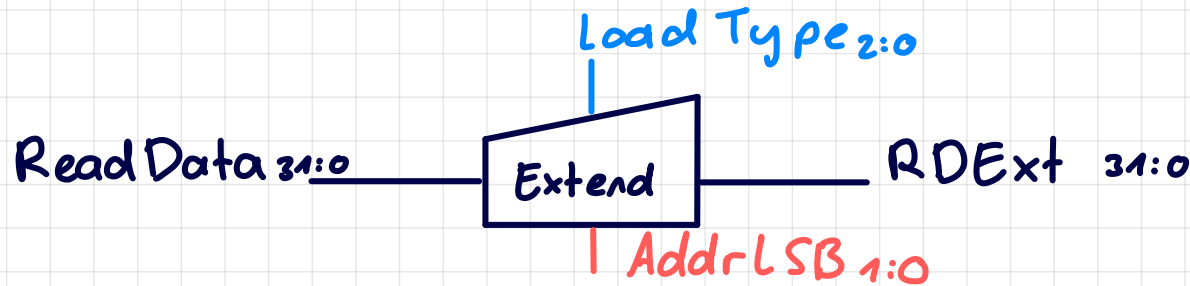


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors



READ DATA - EXTEND ERWEITERUNG



Addr LSB: LSBs der Adresse aus dem ALU-Result um den richtigen Offset im 32-Bit Register auszuwählen

Read Data [31:0]

XX

Addr LSB = 11₂ 10₂ 01₂ 00₂

Erweiterung der Control Unit:

wenn opcode == 0000011 (= load)

dann Load Type $\leftarrow$ func3

sonst Load Type $\leftarrow$ '010' (Alle 32 bit werden durch gegeben, vgl lw instr.)

Extend Architektur:

Load Type	RDExt	Beschreibung	Load Instruction
000	24 x ReadData[7], ReadData[7:0]	Sign Extend 8-bit	lb load byte
001	16 x ReadData[15], ReadData[15:0]	Sign Extend 16-bit	lh load half
010	ReadData[31:0]	—	lw load word
100	24 x 0, ReadData[7:0]	Zero Extend 8-bit	lbu load byte unsigned
101	16 x 0, ReadData[15:0]	Zero Extend 16-bit	lhu load half unsigned

* Werden noch nicht benötigt

INSTRUCTION	Typ	opcode	func3	func7	OPERATION
auipc : rd, up imm	U	001011	—	—	rd = PC + (imm << 12)

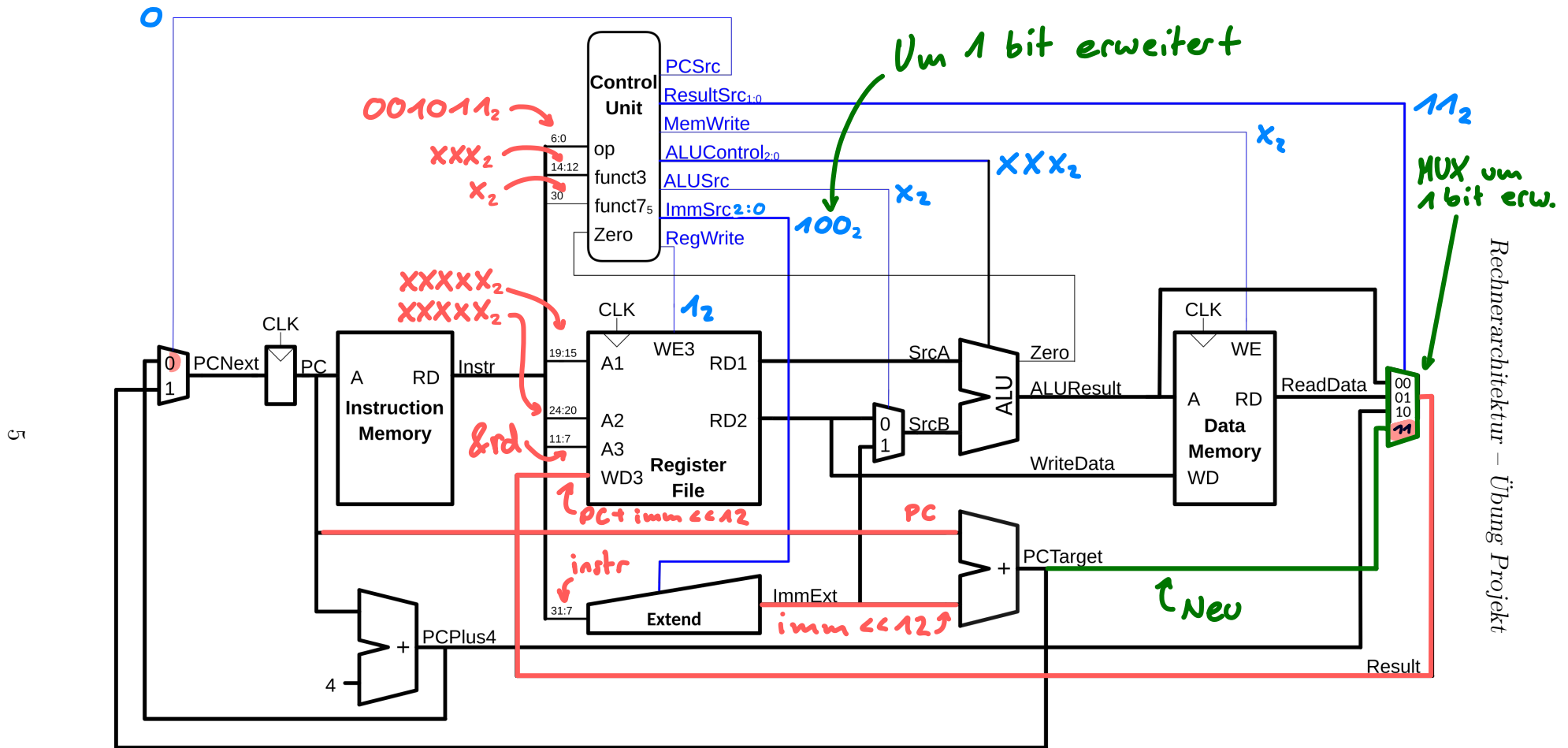


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors

$instr = imm[31:12], 12 \times X$



IMMEDIATES - ERWEITERUNG



instr ... Instruktion aus
Instruction Memory

Imm Src 2:0	Imm Ext 31:0	Typ
000	$20 \times \text{instr}[31], \text{instr}[31:20]$	I
001	$20 \times \text{instr}[31], \text{instr}[31:25], \text{instr}[11:7]$	S
010	$19 \times \text{instr}[31], \text{instr}[31], \text{instr}[7], \text{instr}[30:25], \text{instr}[11:8], 0$	B
011	$12 \times \text{instr}[31], \text{instr}[19:12], \text{instr}[20], \text{instr}[30:21], 0$	J
100	$\text{instr}[31:12], 12 \times 0$	U

Um 1 bit erweitern für U-Typ

bewirkt $\text{imm} \ll 12$